A Framework for Tiny Federated Learning in Resource-Constrained IIoT Environments: A Case Study

Oscar Torres Sanchez, André Rodrigues, Guilherme Borges, *Graduate Student Member, IEEE*, Duarte Raposo, Fernando Boavida, and Jorge Sá Silva, *Senior Member, IEEE*

***Abstract*—The rise of intelligent systems in the Industrial Internet of Things (IIoT) is transforming industrial maintenance by enabling early fault detection and improving operational efficiency. Anomaly detection plays a critical role in this shift; however, centralized training is often impractical due to the large volume of sensor data, limited edge-device capabilities, and constrained networks, particularly in sectors like construction. This work introduces a tiny federated learning (TinyFL) framework for on-device anomaly detection in low-power IIoT nodes, using real-world data from industrial machinery prototypes over constrained networks. Using ESP32-based devices, TinyFL achieves an average F1-score of 97.46%, comparable to centralized and emulated federated setups. Additionally, we propose tiny transfer federated learning (TTFL), a strategy that improves adaptability by selectively updating models only when machines meet specific usage criteria. TTFL combines centralized pretraining with conditional federated updates, outperforming the baseline in 27 of 31 weekly evaluations with an average F1-score of 96.92%, while reducing communication and computation overhead.**

***Index Terms*—Anomaly detection, autoencoder (AE), federated learning (FL), Industrial Internet of Things (IIoT), LoRaWAN.**

Received 26 November 2025; revised 9 January 2026; accepted 30 January 2026. Date of publication 6 February 2026; date of current version 27 April 2026. This work was supported in part by the Foundation for Science and Technology (FCT)–Foundation for Science and Technology through the Ph.D. Grant under Grant 2023.02113.BD; in part by the Centre for Informatics and Systems of the University of Coimbra (CISUC) and European Social Fund, through the Regional Operational Program Centro 2020 under Grant UID/CEC/00326/2020; in part by the FCT Plurianual Funding: Instituto de Engenharia de Sistemas e Computadores de Coimbra (INESC Coimbra) under Grant UID/308; in part by the Recovery and Resilience Plan (PRR), under the Terms of the Call for Proposals [Aviso de Abertura de Concurso (AAC)]-GreenBear, through the Launch of the Research and Development Project Program Aimed at the Development and Implementation of Advanced Cybersecurity, Artificial Intelligence, and Data Science Systems in Public Administration under Grant RE-C05-i08-m04; in part by the Scientific Capacity-Building Program under Grant 04/C05-i08/2024 and Grant 2024.07390.IACDC/2024; in part by the Sistema de Apoio à Criação de Conhecimento Científico e Tecnológico SACCCT–Projetos de Investigação Científica e Desenvolvimento Tecnológico (IC&DT)- Operações Individuais e em Copromoção, Aviso under Grant MPr-2023-12; and in part by the Institute of Systems and Computer Engineering of Coimbra (INESC Coimbra) under Grant COMPETE2030-FEDER-00885800. *(Corresponding author: Oscar Torres Sanchez.)*

Oscar Torres Sanchez and Fernando Boavida are with the Centre of Informatics and Systems of the University of Coimbra (CISUC), 3030-290 Coimbra, Portugal (e-mail: otorres@dei.uc.pt; boavida@dei.uc.pt).

André Rodrigues is with the Polytechnic University of Coimbra, S. Martinho do Bispo, 3045-093 Coimbra, Portugal, also with the Institute for Systems Engineering and Computers at Coimbra (INESC Coimbra), University of Coimbra, Coimbra 3030-290, Portugal, and also with CEOS.PP Coimbra, Polytechnic University of Coimbra, 3045-093 Coimbra, Portugal (e-mail: andre@iscac.pt).

Guilherme Borges is with the Institute for Systems Engineering and Computers at Coimbra (INESC Coimbra), University of Coimbra, 3030-290 Coimbra, Portugal, and also with the Sul-Rio-Grandense Federal Institute (IFSUL), Campus Charqueadas, Charqueadas 96745-000, Brazil (e-mail: guilhermeborges@ifsul.edu.br).

Duarte Raposo is with the Instituto de Telecomunicações, 3810-193 Aveiro, Portugal (e-mail: dmgraposo@av.it.pt).

Jorge Sá Silva is with the Institute for Systems Engineering and Computers at Coimbra (INESC Coimbra), University of Coimbra, 3030-290 Coimbra, Portugal (e-mail: sasilva@deec.uc.pt).

Digital Object Identifier 10.1109/JIOT.2026.3661784

I. Introduction

In THE context of Industry 4.0, the digital transformation of industrial systems is increasingly driven by sensor-rich Industrial Internet of Things (IIoT) infrastructures, edge computing, and machine learning (ML) techniques [1]. When combined with artificial intelligence methods such as anomaly detection, IIoT becomes a key enabler of predictive maintenance and operational efficiency [2].

While many IIoT applications operate in controlled environments with stable local networks based on Wi-Fi or 5G, sectors such as construction, mining, and energy differ substantially. Their equipment is often distributed across wide and dynamic areas where stable power and communication infrastructure are unavailable. In such contexts, low-power wide-area networks (LPWANs) such as LoRaWAN or NB-IoT provide long-range connectivity at minimal energy cost [3]. Unlike fixed local networks, LPWANs support mobile and temporary deployments without requiring complex infrastructure or frequent reconfiguration, making them ideal for fleets of machines that are continuously relocated. These conditions also favor compact, low-cost microcontroller-based devices, which can draw power from existing systems, operate autonomously for extended periods, and integrate easily into legacy machinery. Furthermore, combining a shared microcontroller platform with LPWAN connectivity enables uniform and scalable monitoring across heterogeneous assets, ensuring flexibility, resilience, and cost-effectiveness even in harsh, multisite industrial environments [4].

Under these operational constraints—characterized by limited processing, memory scarcity, and restricted LPWAN bandwidth—integrating ML becomes challenging. Conventional approaches assume abundant resources that are unavailable on low-power nodes, making both training and inference difficult to execute efficiently [5]. Centralized training with large datasets is equally impractical due to constrained connectivity and energy budgets. Federated learning (FL) offers a

2327-4662 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

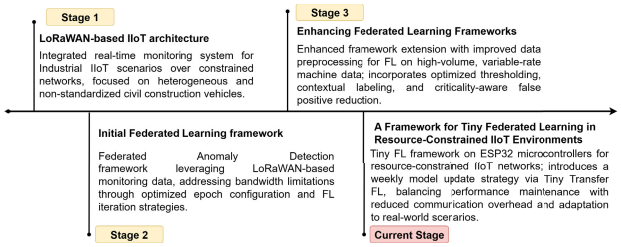


Fig. 1. Research stages in the development of a tiny FL framework for constrained environments.

promising alternative by enabling collaborative model training directly on devices, exchanging only model updates instead of raw data, thus improving privacy and reducing communication overhead. However, FL in highly constrained IIoT contexts still faces open challenges [6]: local training must be optimized for memory and energy efficiency, and transmitting model updates over LPWANs requires compact models to minimize latency. These intertwined limitations motivate the following key research questions.

1) How can anomaly detection models be trained locally under severe resource constraints?
2) What adaptations are necessary to enable efficient model updates over LPWAN networks such as LoRaWAN?
3) How can recurring updates be performed without disrupting industrial operations, overloading the network, or degrading model performance?

Addressing these questions is essential for the practical adoption of FL-based anomaly detection in real-world IIoT environments. To this end, this work presents a novel *Tiny FL (TinyFL)* framework designed to train anomaly detection models directly on resource-constrained IIoT nodes. The framework targets low-power platforms such as the ESP32, operating over LPWANs, and remains adaptable to dynamic and evolving industrial conditions.

This framework builds upon previous research stages illustrated in Fig. 1. It extends the earlier LoRaWAN-based IIoT monitoring architecture by integrating FL mechanisms, neural network design, training configuration (epochs and rounds), and the optimization strategies developed in later stages. While Stages 1–3 were conducted in emulated environments using Python-based ML libraries and frameworks such as Flower, the current stage represents a major shift: the implementation of a TinyFL framework for anomaly detection directly on ESP32 devices through a custom federated scheme optimized for embedded systems. A key innovation is the *tiny transfer FL (TTFL)* strategy, which enables weekly model updates adapted to machine-specific operating conditions. This approach enhances adaptability in dynamic IIoT environments while reducing communication overhead, an essential factor in LPWAN-constrained networks.

The main contributions of this work are summarized as follows.

1) Development of a TinyFL framework for anomaly detection using real industrial data, implemented on ESP32 microcontrollers with communication optimized for constrained resources.
2) Comparative evaluation of TinyFL against centralized and Flower-based FL approaches, considering model performance, training time, memory usage, power consumption, and LoRaWAN communication requirements.
3) Design of the TTFL strategy to support recurring model updates under real-world conditions through selective weekly training. The strategy transfers knowledge from a centralized model and is evaluated across three FL scenarios to identify the most effective configuration under varying machine utilization and data availability.

The remainder of this article is organized as follows. Section II reviews related work in TinyFL and anomaly detection for constrained environments. Section III introduces the proposed framework, detailing data collection, preprocessing, and system design for embedded hardware. Section IV presents the experimental setup, evaluation scenarios, and results, including on-device learning performance, memory use, power consumption, and LoRaWAN viability. Section V describes the TTFL strategy and its weekly evaluation. Finally, Section VI concludes the article and outlines future research directions.

II. Background on TinyFL for Resource-Constrained Devices

A. TinyML Architectures for Anomaly Detection

Recent reviews on industrial anomaly detection ([7], [8], [9], [10], [11]) highlight the predominance of autoencoders (AEs), convolutional neural networks (CNNs), and recurrent neural networks (RNNs) for time-series and sensor data analysis, while noting that most studies overlook TinyML constraints. Complementary surveys such as [12] present integrated views of quantization, pruning, and knowledge-distillation strategies for predictive maintenance.

Experimental works further illustrate these advances. Annapareddy [13] implemented a pruned and quantized CNN on an STM32F407 MCU, achieving 94.7% accuracy with only 42 KB of memory and 32 mW of power. In the unsupervised domain, Long et al. [14] deployed an AE with clustering and dimensionality reduction on an Arduino Nano 33 BLE Sense (3.77 KB footprint, AUC = 0.993), while Lachekhab et al. [15] proposed LSTM-AEs for temporal anomaly detection, reducing false positives (FPs) and improving inference speed sixfold. Additional refinements include hierarchical detectors like in [16] or explainable AE-based models for industrial monitoring on Raspberry Pi platforms like in [17]. Harrabi et al. [18] demonstrated an overcomplete AE reaching an F1-score of 99.23% using less than 17 KB on an Arduino Nano 33 BLE, and Martinez-Rau et al. [19] achieved 96.8% accuracy with under 1.16 μJ per inference on nRF52840 and RP2040 MCUs. Moreover, [20] showed that federated AEs can match centralized performance and outperform classical approaches such as isolation forest (IF) and one-class SVM (OC-SVM).

Collectively, these studies confirm that AEs offer an excellent balance between expressiveness and computational efficiency, with model sizes typically below 20 KB, ideal for ultraconstrained hardware such as the ESP32. However,

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF PLATFORMS/LIBRARIES FOR ON-DEVICE TRAINING IN TINYFL

Platform / Library	Compatible Platforms	On-device Training	Architectures	Known Limitations
TensorFlow Lite Micro	ARM Cortex-M ESP32 Arduino	No	Basic MLPs, CNNs, AE	No dynamic memory allocation, manual model optimization required.
ELL	ARM Cortex-M Raspberry	Limited	MLP, SVM, custom	Trainning via offline compilation, limited support, discontinued official updates.
GLOW	ARM Cortex-A Custom ASICs	Limited	Quantized CNNs	Trainning via ahead-of-time compilation, compilation complexity, needs LLVM toolchain integration.
TVM	ARM Cortex-M RISC-V Raspberry	Yes	CNNs, DNNs	High complexity, initial setup and tuning required.
uTensor	ARM Cortex-M	No	Basic MLP, AE	Limited development, constrained model support.
Genann	C-compatible MCUs	Yes	MLP, AE	Manual training logic, no native support for complex ops.
FANN	ARM Cortex-M RISC-V	No	MLP, AE	Larger memory requirements.
TINN	ARM Cortex-M C99-compatible MCUs	Yes	Basic MLPs	No advanced features.
MicroPython ML Libraries	ESP32, Raspberry ARM Cortex-M Arduino	Yes	Basic MLPs	Slow on-device trainning, execution speed bottleneck, limited numerical libraries.
Keras	ARM Cortex-M ESP32 Raspberry	No	All supported	No native training on device, inference-only

most implementations remain limited to inference, relying on TinyML techniques focused solely on forward execution. This limitation hinders the realization of a fully integrated TinyML framework capable of on-device training. Building upon these insights, our work extends AE-based designs toward *on-device federated training*, enabling adaptive and communication-efficient anomaly detection in LoRaWAN-enabled industrial environments.

B. Platforms, Devices, Libraries, and Frameworks for TinyFL

A wide range of hardware platforms has been explored for executing ML tasks on resource-constrained devices. The landscape is dominated by ARM Cortex-M microcontrollers (e.g., Arduino Nano 33 BLE and Portenta H7) and Espressif systems (ESP32 series), which balance power efficiency with sufficient compute capabilities. Emerging architectures, such as RISC-V-based MCUs and more capable single-board computers (SBCs) like the Raspberry Pi, are also being investigated for edge intelligence [6], [21], [22], [23].

The TinyFL ecosystem encompasses frameworks specifically developed for embedded ML, including TensorFlow Lite Micro, uTensor, ELL, and custom lightweight C/C++ implementations. These solutions vary significantly in hardware support and, crucially, in their capacity for on-device training versus inference-only execution. Table I, adapted from [11], [24], [25], summarizes key frameworks and platforms, highlighting their compatibility with lightweight neural models. This diversity reflects the complex tradeoffs between model expressiveness, computational efficiency, and adaptability required in constrained industrial environments.

Several studies have empirically evaluated ML deployment on these platforms within FL frameworks. Ficco et al. [26] combined FL with transfer learning (TL) using Genann—a minimal C-based library—on heterogeneous devices ranging from the ESP8266 to the Raspberry Pi 3B+, demonstrating how memory constraints directly dictate achievable model performance (86.48% accuracy). Similarly, Giménez et al. [27], [28] analyzed FL on microcontrollers, observing that while increasing the number of training rounds accelerates convergence, it imposes severe penalties on training time and bandwidth usage. They further noted that deeper models, while improving learning capacity, often exceed the available RAM of standard MCUs.

However, a critical limitation persists in the current state of the art: most existing frameworks prioritize computational feasibility and model accuracy without explicitly addressing the interplay between communication frequency, protocol-level overheads, and energy consumption in LPWAN-based IIoT deployments. Building on this foundation and addressing this gap, our study implements a TinyFL framework tailored for industrial anomaly detection on constrained devices such as the ESP32. By adapting a custom neural network using Genann, our approach supports on-device federated training while strictly optimizing learning performance to balance latency and energy constraints of LoRaWAN networks.

C. Communication in TinyFL

Communication remains a primary bottleneck in TinyFL. Several studies ([11], [29], [30], [31]) indicate that communication overhead often dominates the overall training time, posing a major obstacle to practical deployment. This issue is particularly acute in LPWAN systems, where low data rates and strict duty-cycle regulations severely constrain both the frequency and payload size of model updates. To mitigate these limitations, [27] analyzed resource allocation and packet loss during model aggregation, while other works explored weight quantization to reduce transmitted payloads.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

More recently, [32] demonstrated LoRa-based FL using a mesh architecture to enable decentralized participation under severe bandwidth constraints. Similarly, [33] introduced FL-LoRaMAC, a simulation framework that employs PCA-based dimensionality reduction and model pruning, achieving up to a 98% reduction in communication volume. However, while these simulation-based studies show promise, they often overlook the impact of on-device processing time and protocol overhead on the end-to-end training cycle.

Despite these advances, existing approaches frequently lack a critical assessment of the inherent tradeoffs between update latency and energy consumption in real-world LoRaWAN deployments. While payload reduction techniques (e.g., quantization, pruning, and PCA) effectively decrease transmission size, they do not necessarily minimize total energy expenditure [5]. In LoRaWAN networks, energy consumption is driven not only by transmission duration but also by protocol-induced overheads that depend on the device class, including receive windows, acknowledgments, retransmissions, and mandatory idle periods required to comply with duty-cycle regulations. Consequently, frequent model updates, even when highly compressed, can result in excessive energy drain and prohibitive end-to-end latencies, with aggregation rounds potentially spanning several hours or even days in strict regulatory environments [34], [35].

Furthermore, standard FL strategies typically assume that rapid convergence through frequent communication rounds is desirable. In LPWAN-based IIoT systems, however, this assumption is often untenable: increasing the number of rounds to accelerate convergence exacerbates duty-cycle violations and further increases update latency [35], [36]. Crucially, industrial anomaly patterns usually evolve overextended time horizons, on the order of weeks or months, making near-instantaneous convergence unnecessary. Aligning the model update frequency with the physical timescale of system degradation, for instance, through weekly updates, therefore, represents a more effective and realistic strategy. Such an approach avoids operational inefficiencies while ensuring that communication schedules remain compatible with the rigid constraints of LoRaWAN networks [35].

In contrast to prior work that relies on simulations or heavyweight frameworks such as TensorFlow Lite, our approach runs entirely on real microcontrollers using lightweight C-based libraries. Rather than optimizing exclusively for fast convergence, we explicitly address the latency–energy tradeoff by prioritizing feasible and sustainable adaptation. The proposed methodology emphasizes architectural simplification and adaptive, optimized aggregation rounds, demonstrating practical applicability in industrial IIoT environments while maintaining accuracy and energy efficiency under strict LoRaWAN constraints.

D. Federated TL

Recent studies highlight the growing synergy between FL and TL as an effective means to address data scarcity, heterogeneity, and communication constraints in distributed systems [37]. Tang and Jiang [38] proposed a two-stage privacy-preserving framework for wind power forecasting,

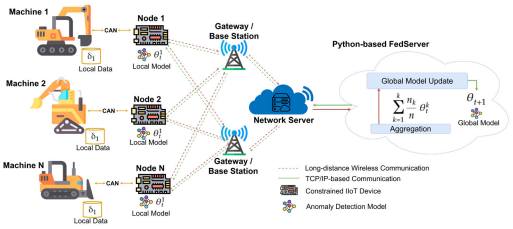


Fig. 2. Overview of the proposed anomaly detection TinyFL architecture for constrained-IIoT environments.

where federated pretraining enabled knowledge sharing across turbines followed by personalized fine-tuning. Their hybrid TCN–GRU model reduced the RMSE by up to 43% and improved accuracy by 27% over standard FL, confirming the benefits of knowledge transfer under limited data and privacy constraints. Likewise, Haripriya et al. [39] developed a hierarchical FL architecture combining pretrained EfficientNet-B0 and TabNet models to enhance fault tolerance and convergence, maintaining over 95% accuracy and reducing communication latency by 25% even when 40% of edge servers failed. Other works [40], [41] fine-tuned pretrained CNNs on distributed clients, achieving near-centralized performance while reducing latency by 30%–40% and accelerating convergence under non-IID conditions.

While these studies demonstrate the potential of TL to improve adaptability and convergence in FL, most implementations target powerful edge devices or simulated environments. Extending these strategies to highly constrained IIoT nodes introduces additional challenges related to computation, memory, and bandwidth. In this context, to the best of our knowledge, this work introduces the first fully integrated TinyFL framework that unifies these aspects within a single design: 1) on-device training on resource-limited hardware; 2) communication-efficient aggregation over LoRaWAN; and 3) adaptive weekly updates aligned with industrial machine behavior using TL. Finally, the proposed framework executes the complete TinyML pipeline—data preprocessing, training, and anomaly detection—directly on the device, maintaining a lightweight architecture optimized for real-world IIoT deployments.

III. Proposed TinyFL Framework

This section presents the core components of the TinyFL framework, a lightweight solution for anomaly detection in highly constrained industrial environments. Fig. 2 provides an overview of the system. Progressively optimized through several research stages (Fig. 1), the framework enables complete on-device execution of data acquisition, preprocessing, and training within a single integrated platform. It incorporates local techniques for data compression, filtering, and aggregation to ensure efficient model training and communication. Designed under the Tiny paradigm, TinyFL aims to preserve detection accuracy while minimizing memory usage, training time, and communication overhead. Its compact AE architecture prevents memory saturation, and all processes, from data handling to model transmission, are adapted to the limitations and duty-cycle constraints of LPWAN technologies

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 3. Data collection stage real-world elements. (a) Prototype installed. (b) Machine running with prototype installed.

such as LoRaWAN. This work is validated using ESP32 devices in a civil construction industry case study, but remains adaptable to other industrial setups with similar hardware or network restrictions. The following subsections describe each component in detail.

A. Data Collection Stage

The data collection process uses resource-constrained IIoT prototypes [Fig. 3(a)] deployed on real construction machinery, as illustrated in Fig. 3. Devices were installed on a multifunction Manitou vehicle (M1), a Terex J-1175 jaw crusher [the red machine in Fig. 3(b)] (M2), and a Doosan DL200 wheel loader (M3). These systems acquire sensor data via the CAN bus (J1939) and analog/digital inputs, which are standardized to the J1939 format and stored locally on SD cards. The devices also support LoRaWAN communication for wireless data transmission. The deployment integrates multiple input sources to improve data resolution for anomaly detection. Processing and training tasks are performed on ESP32-based microcontrollers, specifically the ESP32-S3, which provides sufficient performance for constrained learning tasks when used with optimized models [11], [26].

B. Data Preprocessing Stage

To enable on-device training and anomaly detection in constrained IIoT environments, this study adopts a lightweight preprocessing strategy inspired by [42] and specifically optimized for TinyML deployment. As illustrated in Fig. 4, the proposed pipeline transforms raw heterogeneous data streams (CAN/J1939, analog, and digital) into a normalized, labeled dataset via a sequence of low-complexity operations executed entirely on the microcontroller.

1) *Data Cleaning:* Invalid or malformed J1939 messages are discarded, and stale or inconsistent timestamps are removed to ensure temporal alignment across all sensing sources.
2) *Feature Selection:* To guarantee interoperability across heterogeneous assets, the input stream is reduced to a fixed vector of 12 standardized features common to all machines: engine speed (RPM), oil pressure, battery voltage, engine temperature, fuel consumption, desired RPM, engine load and utilization percentages, friction, ambient pressure, exhaust gas temperature, and exhaust gas pressure.
3) *Frequency Alignment and Aggregation:* As sensors operate at heterogeneous sampling rates, data are aligned by grouping samples according to the lowest common acquisition frequency. Within each time window, a *mean representation* is computed for each feature, providing noise attenuation while maintaining minimal computational overhead.
4) *Anomaly Label Generation:* A lightweight hybrid labeling strategy is applied to support supervised evaluation and semi-supervised training. First, *technical labeling* marks samples as anomalous when predefined operating limits are exceeded, which are defined through operating manuals, manufacturing data, or expert operator experience. Second, *start-up labeling* explicitly marks the first 10 min of machine operation as nonanomalous warm-up periods, preventing the misclassification of transient cold-start behaviors.
5) *Feature Standardization:* Finally, all features are normalized using Z-score standardization (mean and standard deviation). To comply with memory constraints, normalization parameters are computed incrementally using a streaming algorithm, avoiding the need to store large historical buffers.

TABLE II
MACHINE DATASETS

Metric/Machine	Multifunction M1	Jaw Crusher M2	Wheel Loader M3
Training	35,004	3,553	16,965
Test	7,449	754	3,698
Validation	7,523	774	3,604

This preprocessing pipeline ensures that the resulting training data remain compact, consistent, and reproducible across devices. Table II summarizes the postprocessed datasets per device, including the number of samples containing all selected features. The following subsections detail the federated training process and the integration of local models within the proposed framework.

C. FL Stage

This section details the implementation of FL within the proposed IIoT architecture. It describes the end-to-end process, including on-device training, model evaluation, and anomaly estimation. Key components such as local weight extraction, model update transmission, and the centralized aggregation via the FedAvg algorithm are addressed. Additionally, the mechanism for distributing the updated global model back to the devices is explained. The federated process operates on the standardized and labeled data described in the previous subsection.

1) AE Initialization on ESP32-Based Devices: As previously noted, this study employs an AE architecture for anomaly estimation. The implementation relies on Genann,

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

a lightweight C-based neural network library designed for training and executing artificial neural networks (ANNs) in resource-constrained environments [43]. In contrast to high-level frameworks such as TensorFlow, which encapsulate memory management and computational routines, Genann provides low-level control over these processes, an essential property for enabling complete on-device training and inference on microcontroller platforms, such as the ESP32-S3. While Genann natively supports basic backpropagation with sigmoid and linear activation functions, the library was extended and adapted to align with the requirements of the proposed TinyFL framework. The present implementation builds upon the microcontroller-oriented improvements introduced by [26], including CSV parsing, floating-point conversion, and weight export functionalities. Beyond these enhancements, this work introduces additional mechanisms to enable on-device anomaly detection through an AE-based structure and embedded training process. The principal adaptations are summarized as follows.

1) The Genann library was extended with a customized **tanh** activation function optimized for low-power execution. It employs a precomputed lookup table (LUT) of 4096 entries, occupying about 16 kB in memory, well within the ESP32's PSRAM. During initialization, the LUT precalculates $\tanh(x)$ over a bounded domain, and evaluations retrieve values directly from memory in O(1) time. Inputs beyond the limits are clipped to ± 1 to prevent overflow. This design preserves the zero-centered benefits of tanh, improves convergence stability, and reduces activation cost during on-device AE training.
2) *Optimized Line-by-Line CSV Reading From SD Cards:* To train on devices with limited memory, data is processed sequentially from the SD card instead of being loaded entirely into RAM. The data reading function uses a fixed 256-byte buffer, tokenizes each line, converts the values to floats on the fly, and immediately sends them to the Genann train, freeing the read value from memory. For the AE, the input and output vectors are replicated for training. This streaming approach allows training on datasets larger than the available SRAM while maintaining stable execution. The function is also reused for inference and anomaly detection.
3) *Support for Auxiliary Input Files*, including variable anomaly thresholds, anomaly labels, and contextual labels, enabling semi-supervised evaluation.
4) *Parameterized FL Configuration*, allowing flexible control over the number of communication rounds, as explored in [20] and [42].
5) *FedAvg Aggregation Support*, allowing devices to transmit the number of local training samples to enable correctly weighted model aggregation.
6) *On-Device Computation of Domain-Specific Metrics*, including TNR, TPR, F1-score, precision, and accuracy. Existing processing routines compute MSE values, label anomalies, and reclassify instances when contextual information is available.

TABLE III
AE PARAMETERS FOR TRAINING EXECUTION

Model	Parameter	Value
AE	Input / Output Layer	12
	Encoder / Decoder Hidden Layer	64
	Bottleneck Layer	2
	Activation function	Tanh
	Threshold Value	Adaptable by Feature
Trainning Configuration	Epoch	5
	FL rounds	16

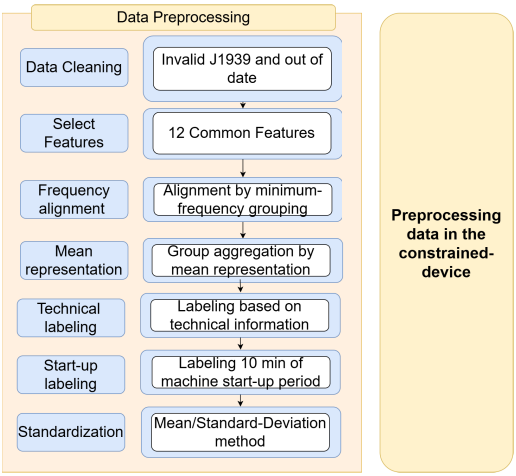


Fig. 4. Data preprocessing stage diagram flow.

Table III summarizes the structure of the AE used for anomaly detection, along with the configuration of local epochs and FL rounds.

2) On-Device AE Training Procedure in an FL Context: Once the AE is initialized on the ESP32 devices, each node executes its local training procedure as part of the FL process. Fig. 5 depicts the complete device-side workflow, explicitly highlighting mechanisms for weight segmentation, duty-cycle compliance, and loss recovery.

The process begins with configuring local hyperparameters and initializing the file system. To accommodate the limited RAM available on microcontroller-class devices, training data are streamed from the SD card rather than loaded entirely into memory. Before entering the training loop, the device checks for updated global model weights from the server. If available, these weights are loaded to enable incremental learning; otherwise, the device retains its last valid local model or applies random initialization during the first communication round.

Local training proceeds by sequentially processing samples and updating the model parameters (weights) via backpropagation using the Genann library. Upon completion of local optimization, the complete set of model weights is serialized into a contiguous representation and subjected to an explicit segmentation process. To comply with LoRaWAN payload limitations, the serialized model is partitioned into N fragments, where each fragment encapsulates a subset of model weights and is sized according to the maximum transmission unit (MTU) associated with the current spreading factor, following the approach proposed in [20]. Between consecutive

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

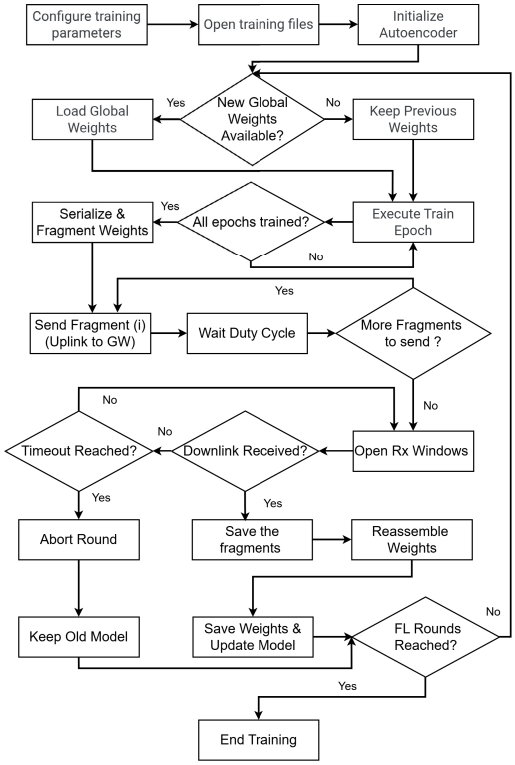


Fig. 5. LoRaWAN-enabled training algorithm in FL.

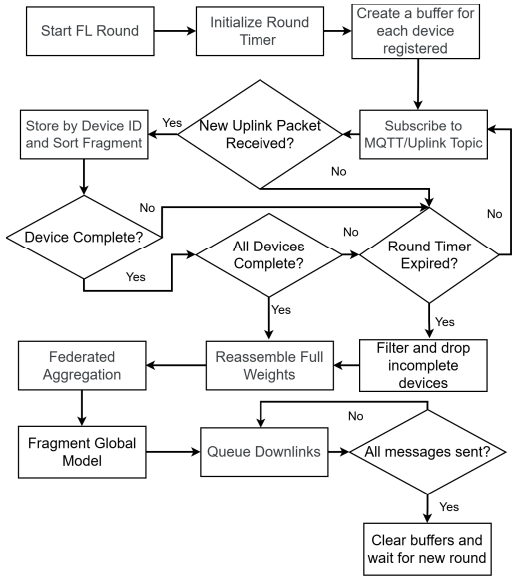


Fig. 6. FL server algorithm in LoRaWAN context.

fragment transmissions, the device enters a wait state to strictly enforce duty-cycle constraints in the ISM band.

After transmission, the device opens its receive windows to capture the aggregated global model. Given the inherent unreliability and latency of LPWAN communications, a timeout-based recovery mechanism is implemented. If the expected downlink is not received within the designated receive window, an *Abort Round* event is triggered: the current iteration is discarded, and the device reverts to its previously validated model to preserve system stability. Conversely, if the downlink is successfully received, the fragments are reassembled, and the local model state is updated. The device then verifies whether the predefined number of federated rounds has been reached; if not, the workflow continues seamlessly into the next training epoch.

3) FL Server and Aggregation Strategy: On the server side, the FL coordinator operates within the LoRaWAN ecosystem to manage asynchronous updates from distributed edge devices and execute the aggregation process. Fig. 6 illustrates the reliability-aware server workflow, explicitly capturing fragment handling, loss tolerance, and early aggregation logic.

At the beginning of each communication round, the server initializes a *Round Timer* and queries the registry of active devices associated with the corresponding The Things Network (TTN) application. For each registered device, a dedicated buffer is allocated to store incoming model fragments. The server then subscribes to the uplink topic to receive decoded messages forwarded by the TTN infrastructure.

Due to payload constraints and network latency, LoRaWAN messages arrive asynchronously and in fragmented form. Upon receiving a packet, the server stores it in the appropriate device buffer. It evaluates two completion conditions: 1) *device completion*, which verifies whether all expected fragments from a given device have been received and 2) *global completion*, which checks whether all registered devices have completed their uploads. If the global completion condition is satisfied, the server immediately proceeds to reassembly and aggregation, bypassing the remaining wait time to reduce end-to-end latency.

If one or more devices fail to complete transmission within the allotted window, the Round Timer acts as a fallback mechanism. Upon expiration, an *Integrity Check* is performed: devices with incomplete or inconsistent buffers are excluded from the current round to prevent misaligned updates from corrupting the global model.

Valid model updates are then reassembled and aggregated using the federated averaging (FedAvg) algorithm. Each participating device i contributes a local update θ_i^t weighted by its number of training samples n_i, producing the new global model θ^{t+1} as defined in the following equation:

$$\theta^{t+1} = \sum_{i=1}^{N} \frac{n_i}{\sum_{j=1}^{N} n_j} \cdot \theta_i^t. \tag{1}$$

Finally, the updated global model θ^{t+1} is segmented into downlink-compatible fragments to satisfy LoRaWAN payload constraints. These fragments are inserted into a prioritized downlink queue, where the gateway schedules transmission based on network availability. Once all downlink messages have been transmitted, temporary buffers are cleared, and the server transitions to the next communication round. This iterative process repeats for the configured number of federated rounds (e.g., 16), enabling robust model evolution despite packet loss, asynchronous arrivals, and strict LPWAN constraints.

4) Anomaly Estimation: Global updated AEs require a complementary mechanism to identify anomalies based on their reconstruction error. This approach relies on setting an MSE-based threshold to determine whether a data point

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

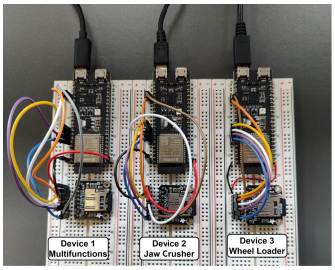


Fig. 7. ESP32-based microcontroller used on the TinyFL framework.

is anomalous. The threshold selection process, introduced in [42], iteratively searches for the optimal F1-score. After training, with the AE outputs is computed the MSE per feature is computed, and a threshold is chosen using a percentile of these values. Data points exceeding this threshold are labeled as anomalies, and a new F1-score is calculated. The ground-truth labels are derived from technical manuals and some expert operator knowledge, providing domain-specific criteria for distinguishing normal and anomalous behavior. If this score is lower than the initial one, a different percentile is chosen, and the process repeats until the optimal F1-score is found. This final threshold is saved for future use.

In the context of this process, several important steps must be followed to compute the evaluation metrics, such as the F1-score, directly on the device. First, the device processes the file containing the error calculations mse and the previously determined optimized threshold values. Using this information, it estimates anomalies for each feature. After calculating the anomaly scores, it is necessary to synchronize this data with the contextual labels derived from the machine's operational information. This step ensures that data points labeled as anomalies during the model's warm-up phase are correctly unlabeled, as they are not considered true anomalies. Next, the 12-feature sample is evaluated by applying a logical rule: if at least one feature is identified as anomalous, the entire sample is labeled as an anomaly. This binary anomaly label is then compared with the ground truth labels, enabling the evaluation of the model despite the AE's unsupervised nature.

IV. Experimental Setup and Evaluation

The evaluation compares the proposed TinyFL framework with its emulated counterpart to assess training feasibility and effectiveness on resource-constrained devices within LoRaWAN contexts. The ESP32-S3 microcontroller, with a dual-core Xtensa LX7 processor at 240 MHz, 512 KB SRAM, and variants supporting 8 MB PSRAM and 16 MB flash (see Fig. 7), is used for lightweight ML tasks. Training leverages C-based libraries like Genann, while a Python server manages model aggregation. Further evaluation scenarios and implementation details follow in subsequent sections.

It is important to note that the data used in this study are inherently non-IID. The datasets were collected over 12 months from multiple industrial machines that did not operate simultaneously nor under identical working conditions, despite being deployed on the same construction site. Differences in operational schedules, load profiles, environmental conditions, and machine usage patterns result in heterogeneous data distributions across devices. Consequently, the proposed TinyFL framework is evaluated under realistic non-IID conditions representative of real-world IIoT deployments, rather than assuming artificially homogeneous data.

TABLE IV
TinyFL Evaluation Metrics

Metric	Accuracy	Precision	F1 Score	TPR	TNR
Mean	95.51	97.01	97.46	97.99	79.46
Std	2.58	3.40	1.54	0.53	14.68

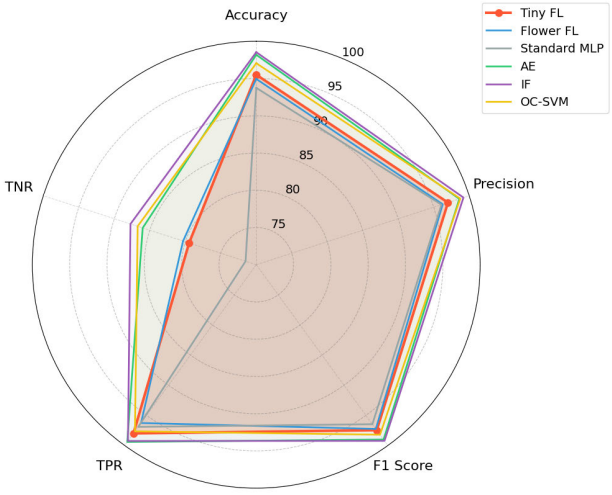


Fig. 8. Performance comparison across different training approaches.

Our work as evaluation metrics uses labels normal values as "positive" and anomalies as "negative." True positives (TPs) are correctly identified normal values, FPs are missed anomalies, true negatives (TNs) are correctly detected anomalies, and false negatives (FNs) are false alarms. Although the confusion matrix is not shown, these values are used to calculate the reported metrics. Model performance is assessed using accuracy, precision, specificity (TNR), sensitivity (TPR), and F1-score, providing a comprehensive measure of anomaly detection effectiveness.

A. Performance Assessment of the TinyFL Framework

After establishing on-device evaluation metrics, experiments assessed the TinyFL framework's performance with five local epochs and 16 aggregation rounds. Experiments were repeated using random dataset splits preserving class distribution (Table II). Table IV shows the results: the TinyFL framework achieved 95.51% accuracy (±2.58 SD) and 97.46% F1-score (±1.54 SD), with a TPR of 97.99% (±0.53 SD), indicating strong anomaly detection. The TNR averaged 79.46% (±14.68 SD), affected by class imbalance and FNs, which is typical in anomaly detection and acceptable for this application.

As illustrated in Fig. 8, the comparison includes TinyFL (on-device), Standard MLP (on-device), Flower FL (emulated), and centralized models (AE, IF, and OC-SVM). The centralized IF approach achieves the highest performance, thanks to the complete availability of data and unrestricted computational resources. Closely behind, approaches such as

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EXECUTION TIME AND ENERGY CONSUMPTION PER ML TASK ON EACH MACHINE

Task	Machine	Time (s)	Energy (J)
Training One Epoch	M1	63.15	5.96
	M2	6.32	0.57
	M3	30.22	2.39
Training One round	M1	314.51	29.66
	M2	31.74	2.87
	M3	157.42	11.94
Inference Single Input	M1	2.4E-3	3.6E-4
	M2	2.4E-3	3.6E-4
	M3	2.4E-3	3.5E-4
Anomaly Estimation Full Test Set	M1	16.29	4.38
	M2	16.50	4.31
	M3	16.49	4.28

AE and OC-SVM show similar performance with a balance between F1-score and precision.

Federated approaches show a moderate performance gap relative to centralized baselines, as expected from their distributed training and limited local data. Nevertheless, TinyFL and Flower FL maintain competitive results across all metrics. Notably, TinyFL, executed on constrained ESP32-S3 devices, matches or surpasses Flower FL in F1-score, demonstrating the effectiveness of its lightweight AE-based design for anomaly detection. The standard MLP with a single hidden layer exhibits the lowest performance, confirming that the AE topology in TinyFL provides superior reconstruction-based estimation.

Further analysis shows that the main divergence lies in TNR, where centralized models, particularly IF, perform better. Federated methods exhibit lower stability in this metric due to class imbalance and aggregation variability. However, TinyFL and Flower FL remain closely aligned with centralized baselines in TPR and F1-score, showing that on-device federated training can deliver robust anomaly detection with only a slight increase in false alarms. Overall, while centralized models remain the benchmark, TinyFL achieves a practical balance between feasibility and accuracy. It approaches centralized baselines on key metrics such as F1-score and TPR, while remaining fully deployable in ultraconstrained IIoT environments with limited computation and communication resources.

B. ML Tasks Performance on Constrained Devices

Given their constrained nature, it is essential to assess the feasibility and efficiency of executing ML tasks directly on ESP32-based devices. These devices train models using the algorithm described in Section III-C2, employing an AE implemented with the Genann library. Table V summarizes average execution times and energy consumption for key ML tasks. Using identical hardware, one epoch of training takes 63.15, 6.32, and 30.22 s, with corresponding energy usage of 5.96, 0.57, and 2.39 J for M1, M2, and M3, respectively, showing that dataset size and runtime strongly influence energy demand. With five local epochs before FL communication, the M1 requires over 5 min and 29 J per round, while the M3 and M2 need about 3 and 1 min, consuming 2 and 1 J. These timing differences create synchronization challenges, as the slowest device defines the round duration and becomes the device that delays aggregation on the server. During anomaly estimation, all devices perform similarly, averaging 16 s and 4.3 J, whereas single-input inference takes only 2.3 ms and 0.36 mJ, enabling real-time detection. In the highest-power scenario (M1), with five epochs over sixteen rounds, the ESP32 consumes 468 J in 314 s (93 mW average). A 3.7 V, 1000 mAh LiPo battery (13320 J) could power it for about 40 h, sufficient for 28 weekly training cycles or over 2200 single epochs. These results confirm the framework's energy efficiency and suitability for long-term deployment in low-power IIoT environments, even under limited energy availability.

C. On-Device Memory Requirements

Regarding memory requirements, the resulting neural network, defined according to Table III and representing an AE, comprises 152 neurons organized in a 12–64–64–12 topology. The number of trainable parameters (W) corresponds to the total number of weighted connections between layers, including bias terms, and is given by

$$W = (n_{\text{in}} + 1)\, n_{h1} + (n_{h1} + 1)\, n_{h2} + (n_{h2} + 1)\, n_{\text{out}} \quad (2)$$

where n_{in}, n_{h1}, n_{h2}, and n_{out} denote the number of neurons in the input, first hidden, second hidden, and output layers, respectively, like ($n_{\text{in}} = 12$, $n_{h1} = 64$, $n_{h2} = 64$, $n_{\text{out}} = 12$). Therefore, the AE used in this study consists of 152 neurons and 5772 trainable weights, consistent with the fully connected implementation defined by Genann. Considering a floating-point representation in the ESP32 environment, the memory required to store the model parameters corresponds to approximately 22.54 kB.

The training function uses preallocated PSRAM to store weights, leaving 8338 kB free after allocation. Combined with the ESP32-S3's 512 kB SRAM, this provides 8440.55 kB for dynamic allocation, which remains stable, indicating efficient memory management without fragmentation. All collected and preprocessed data are stored on an external SD card, as dataset sizes exceed internal memory. Training datasets require 8093 kB for the multifunction machine, 3917 kB for the wheel loader, and 808 KB for the jaw crusher. Data are processed sequentially, loading one sample at a time. Each training input is a 12-value matrix (0.04 kB), and due to the AE's reconstruction process, a copy is temporarily stored for error comparison, adding 0.04 kB. Test and validation datasets (2747 kB each) also reside on the SD card. During estimation, memory usage is minimal: 0.096 kB for input and errors, 0.01 kB for labels. The threshold determination phase requires 0.04 kB to store optimal values used for classification.

D. LoRaWAN Viability for TinyFL Model Transmission

Although the experiments were not performed on a LoRaWAN public network or the private network used for data collection, they were conducted under controlled conditions that replicate a LoRaWAN environment, including the regulatory constraints of a private deployment. Considering the duty-cycle limitations, payload segmentation, and spreading factors, this section evaluates whether the obtained

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

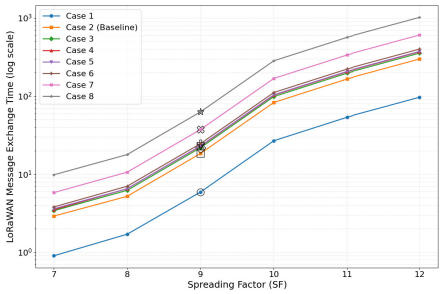


Fig. 9. LoRaWAN messages exchange time.

transmission times align with realistic operational scenarios. Additionally, it examines the impact of model complexity by testing multiple AE topologies with varying network depths and hidden-layer widths, while preserving the overall AE structure, to assess their effect on training time.

The configurations explored ranged from Case 1 to Case 8. Case 1 defined a compact structure with two hidden layers of 32 neurons (1,868 parameters, 7.3 KB), while Case 2, the reference model used in previous experiments (Section IV-A), expanded the hidden layers to 64 neurons (5772 parameters, 22.5 KB). Case 3 adopted a more extensive 12–64–32 architecture with a mirrored decoder, totaling 216 neurons and 26.8 KB. Case 4 followed the same principle, introducing an additional 16-neuron encoding layer (12–64–32–16), resulting in 27.9 KB. Case 5 extended this design further (12–64–32–16–8), maintaining symmetry through mirrored decoders. Conversely, Cases 7 and 8 simplified the depth but increased the initial hidden layer width to 96 and 128 neurons, yielding model sizes above 45 and 77 KB, respectively.

Fig. 9 and Table VI illustrates the estimated time required to complete an FL session across these configurations under different LoRaWAN spreading factors (SF7–SF12). At SF9, the configuration used during the data collection phase, message exchange times range from under 5 h (Case 1) to over 70 h (Case 4). In the reference Case 2, the transmission time is approximately 18–20 h. Although this duration may appear long, it remains feasible within weekly training schedules by exploiting machine downtime (e.g., during weekends). Extremely compact models, while faster to train and communicate, may compromise anomaly detection accuracy. Conversely, more complex architectures impose excessive communication and airtime demands. Thus, Case 2 emerges as a balanced configuration, maintaining sufficient representational capacity for reliable anomaly detection while keeping training and communication durations compatible with realistic operational constraints in LoRaWAN-enabled IIoT environments.

Another challenge lies in synchronizing messages, as weights may arrive at different times. Devices do not receive immediate confirmation from the server, so the number of expected packets must be adjusted according to the fixed total number of weights. This allows the server to track how many weights should be received, even if they arrive asynchronously. Strict coordination is essential for the FedAvg function since each weight must align with the correct position in the model. Once the global model is ready, the gateway should forward it to the devices via LoRaWAN. Our implementation uses class C devices, which, despite higher power usage, support nearly continuous downlink reception, suitable when the device is connected to the machine's battery. The framework can also support class B devices with scheduled reception, while class A is less suitable due to its strict uplink-triggered downlink windows.

V. Weekly Model Update Strategy for TinyFL

This phase implements a weekly training and evaluation strategy involving three construction machines. The goal is to assess weekly the anomaly detection performance of an AE model trained using the TinyFL framework proposed in this work under realistic and dynamic industrial conditions. This new strategy seeks to execute the framework, establishing predefined conditions that determine whether a machine has accumulated enough operational data to justify training in a specific week. Doing so avoids unnecessary weight exchange in weeks where machines have seen limited or no activity, optimizing communication and computational resources.

A. Weekly Participation Rules for FL Under Real-World Constraints

Over 54 weeks of data collection, machine activity varied: multipurpose machines operated regularly, while single-purpose ones were often inactive. Some weeks produced insufficient data for training or model updates over LoRaWAN. To handle these real-world constraints, we established criteria to determine when a machine is eligible for training or testing.

1) The machine must have been active during the week under analysis.
2) After preprocessing, the machine must have at least 100 data entries for training and at least 30 for testing. The dataset is split 70% for training and 30% for testing.
3) For the FL process to occur in a given week, at least two machines must meet the training data requirement.
4) If a machine meets the training requirement but no other peers qualify for aggregation, it performs only local testing.
5) Machines that do not meet the training requirement or no FL process is executed, store the unused data, and defer training until conditions are met in future weeks.
6) All machines perform testing using the most recently available model, which may have been generated in the previous week.

Once the criteria for determining whether a given week is valid for each machine have been established, we classified the 54-week period to identify which weeks were suitable for training and which could only be used for testing. As a result, the analysis showed that 34 of the 54 weeks met the minimum requirement of having at least one machine capable of running model evaluation. Specifically, among these, the multifunction (M1) machine qualified in 30 weeks, the wheel loader (M3) in 30 weeks, and the jaw crusher (M2) in only 11 weeks. To better visualize the temporal distribution of data across the machines, Fig. 10 presents a heatmap illustrating the total number of data samples generated each week for each device.

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

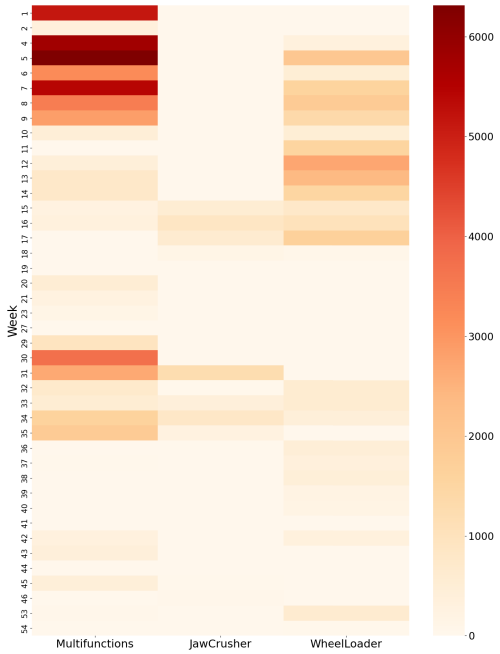


Fig. 10. Weekly data distribution.

This visualization highlights fluctuations in machine utilization over time. In particular, data availability is uneven, with some machines being more active during certain weeks while others remain inactive. Therefore, these disparities reinforce the importance of designing an adaptive and efficient strategy for updating the model only when sufficient and meaningful data are available.

B. TTFL Strategy

After characterizing the data distribution and defining the implementation challenges, this section introduces the TTFL strategy, which integrates FL and TL to enable periodic model updates in constrained environments.

The TL process provides a robust pretrained model that allows devices to perform fine-tuning once sufficient local data is available. This approach mitigates underfitting during the initial training phases and reduces the frequency of global updates, particularly relevant in LoRaWAN scenarios where communication efficiency is critical. It also facilitates adaptation to evolving operational conditions while maintaining stable anomaly detection performance. To generate the initial centralized model, training data from all three machines are used, selecting the first four valid weeks from each dataset. The combined data are divided into 70% for training, 15% for validation, and 15% for testing. The model is trained for 80 epochs using the same AE architecture defined in Table III and the threshold-based anomaly detection method described in Section III-C4. The resulting weights, denoted as W_{transfer}, serve as the initialization for subsequent training within the FL process.

C. Evaluation Scenarios

Now that the importance of TL has been established within the weekly update framework, we evaluate its performance across multiple scenarios. Each scenario is designed to implement a TTFL process over 54 weeks, under varying operational conditions, as detailed below. In all cases, machines must adhere to the participation criteria for training and testing outlined in Section V-A. Based on this premise, the following scenarios are defined as follows, and the overall process for each scenario is illustrated in Fig. 11.

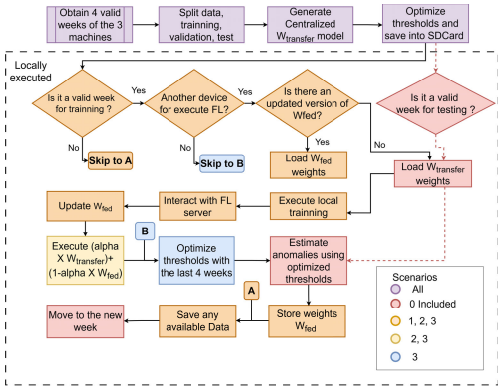


Fig. 11. Scenarios diagram flow.

1) Scenario 0: This initial scenario serves as a baseline for comparison with the subsequent ones. The core premise is that once the W_{transfer} model has been defined, no further fine-tuning or weekly updates via FL are applied. Instead, the focus remains solely on testing, provided the machine meets the testing criteria for a given week. This approach aims to evaluate whether continuous model refinement strategies offer performance advantages over relying exclusively on an initial centralized TL model, so devices use W_{transfer} only for testing for all 54 weeks.

2) Scenario 1: In this scenario, the initial model is generated using data from the first four weeks, producing a set of weights denoted as $W_{transfer}$ along with an optimized set of thresholds. These weights are stored in the memory of each device.

Based on predefined criteria, each device first determines whether it has sufficient data to qualify the current week as valid. If the week is deemed valid for training, the device checks if at least one other device has also reported its training readiness. If multiple devices meet this condition, the training process begins.

Each device initializes its model with $W_{transfer}$ for the first local training iteration and begins training. After completing their respective epochs, the devices iteratively send their updated weights to the server, continuing this process until the federated training cycle is complete.

At the end of the federated process, the final aggregated weights, denoted as W_{fed}, are used for anomaly detection within the same week and are stored for future training. Training no longer starts with $W_{transfer}$ in subsequent valid weeks but with the most recent W_{fed}. This ensures that $W_{transfer}$ is only used in the initial phase and is not applied in later iterations.

If a device lacks sufficient data to validate the current week, it stores any collected information and waits for a future valid week. If a device has enough data to qualify the week as valid but no other device is available for federated training, it stores

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
MESSAGE AND TIME REQUIREMENTS PER SF

SF	7	8	9	10	11	12
Messages	1664	1664	3212	7243	7243	7243
Min. Hours	2.9	5.2	18.4	82.9	165.6	298.4

the training data for future use. The device only tests using the most recent W_{fed} model in such cases.

3) Scenario 2: This scenario follows the same procedure as Scenario 1, with a key distinction: the weights W_{transfer} are continuously integrated throughout all weeks. After each FL cycle, devices compute a weighted combination of W_{transfer} and the newly aggregated weights W_{fed}, using the following formula:

$$W_{\text{fed}} = (\alpha \times W_{\text{transfer}}) + (1 - \alpha) \times W_{\text{fed}_0}. \quad (3)$$

With α set to 0.7, this configuration ensures that the updated model W_{fed} reflects a stronger influence from the centralized model W_{transfer}. At the same time, keep updating from the new W_{transfer} model. This blended approach provides continuity between training rounds, using weights that incorporate both prior knowledge and recent adaptations. It helps stabilize model performance amid evolving data distributions by maintaining a controlled balance between generalization and local adaptation.

4) Scenario 3: This scenario extends Scenario 1 by introducing a local optimization step for anomaly-detection thresholds, thereby reducing reliance on static parameters derived from the initial centralized W_{transfer} model. In this approach, each device uses its stored weekly data for two distinct purposes: training data supports FL updates (when conditions are met), while test data is repurposed to refine detection thresholds dynamically. Specifically, the system employs a sliding window strategy that aggregates test data from the four most recent valid weeks, excluding the current analysis week. This four-week horizon was selected to align with the typical operational periodicity of construction machinery, where behavioral shifts driven by project phases, load profiles, and environmental conditions occur on a monthly timescale. This choice is empirically supported by the cyclic patterns shown in Fig. 10 and is corroborated by operator feedback on maintenance routines. From a statistical perspective, this duration strikes a critical balance: shorter windows risk excessive sensitivity to transient fluctuations (leading to more FPs), whereas longer windows may delay the detection of genuine anomalies. By updating thresholds weekly, the system captures meaningful behavioral trends while filtering short-term volatility.

D. Results Overview

After evaluating the different scenarios, it is essential to analyze the performance of the proposed TTFL strategy. Scenario 0 serves as the reference case, where only the centralized model W_{transfer} is applied weekly for testing. The average F1-scores across all weeks are summarized in Table VII, labeled "S0." Device identifiers are defined as follows: 1—multifunction machine, 2—jaw crusher, and 3—wheel loader. The centralized model performs well during the first 30 weeks; however, in weeks 31 and 32, a marked drop in F1-score occurs, particularly for devices 2 and 3, coinciding with a long interval between training rounds (last updated in week 18) and the delayed arrival of new data. During this period, sensor readings diverged from the original training conditions due to temporary operational changes, a frequent occurrence in construction environments. As a result, the S0 model becomes outdated and struggles to adapt to the evolving data distribution. Fig. 10 shows that machine 3's increased activity further accentuates these deviations, contributing to model degradation. Similar declines in weeks 42 and 43 highlight that temporary operational shifts can invalidate pretrained models. These findings emphasize the need for adaptive learning strategies beyond static TL to maintain consistent performance and reliable anomaly detection in dynamic IIoT environments.

Based on the benchmark results from Scenario 0, we now analyze the performance of Scenarios 1–3. Table VII summarizes the outcomes, presenting the F1-score for each scenario, highlighting the higher value each week for all devices. Several key insights emerge from the analysis. Scenario 1 demonstrates the most consistent performance, particularly during the first 30 weeks. In this period, Scenario 1 outperforms the baseline in 17 out of the 18 weeks analyzed. These early weeks are characterized by the predominant participation of machines 1 and 3, with machine 3 showing notably better performance, thereby contributing significantly to the overall improvement. In contrast, Scenarios 2 and 3 exhibit consistent underperformance during these initial weeks. The F1-score across all early weeks suggests that combining a weighted model with the centralized model and applying threshold adjustments using a sliding window does not lead to better performance than the static centralized baseline (S0).

After week 30, Scenario 3 shows superior performance in 7 of the remaining 15 weeks, with notable gains of 40 and 34.9 points in weeks 31 and 32, respectively. Devices 1 and 2 exhibit improvements of up to 54.2 points, highlighting the benefits of adaptability. These results demonstrate that combining frequent model updates with centralized and adaptive threshold adjustments greatly enhances performance, especially when new devices are introduced or machine behavior deviates from normal operation. Overall, the hybrid TTFL strategy achieves robust performance across dynamic and heterogeneous scenarios while reducing communication frequency with the aggregation server, a key advantage in constrained networks.

By applying this strategy and executing model updates only when necessary, the system maintains high anomaly detection accuracy while conserving computational and communication resources. For M1, local training time and energy consumption decrease by approximately 50%, while M2 and M3 achieve reductions of 45.45% and 46.66%, respectively. Over 52 weeks, these optimizations yield cumulative savings of 75 482 s and 7,118.45 J for M1, 2,539.2 s and 229.6 J for M2, and 35 262.08 s and 2,674.56 J for M3, along with nearly 257 h of avoided LoRaWAN transmissions. These results confirm the viability and efficiency of the adaptive

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
WEEKLY F1-SCORE SCENARIO 0 COMPARED WITH PERFORMANCE IN SCENARIOS 1–3

		Week																
Dev	**S**	**6**	**7**	**8***	**9***	**10***	**11**	**12***	**13***	**14***	**15***	**16***	**17**	**18***	**20**	**21**	**23**	**29**
All	**0**	98.7	97.7	98.3	97.9	99.0	97.3	98.5	99.4	98.1	98.5	98.1	98.2	95.7	99.6	99.5	98.2	98.4
	1	**98.7**	**97.7**	**98.4**	**98.2**	**99.0**	**98.2**	**98.7**	**99.6**	**98.5**	**98.8**	**98.6**	**98.7**	**97.0**	**99.8**	**99.5**	**99.0**	**98.4**
	2	98.7	97.7	96.3	97.7	98.3	97.2	97.8	98.4	96.9	97.0	97.1	97.6	90.0	98.6	99.7	98.6	97.8
	3	97.8	96.5	71.7	95.9	95.8	95.9	98.7	98.9	97.3	98.1	97.0	97.8	80.4	93.0	98.9	99.0	96.4
1	**0**	98.7	97.7	97.8	97.5	98.9	100.0	100.0	99.5	97.8	99.1	99.3	-	-	99.6	99.5	98.2	98.4
	1	98.7	97.7	98.0	97.7	99.0	100.0	100.0	99.5	98.5	99.3	99.2	-	-	99.8	99.5	99.0	98.4
	2	98.7	97.7	95.1	97.4	98.1	100.0	99.9	98.5	96.5	96.1	98.6	-	-	98.6	99.7	98.6	97.8
	3	97.8	96.5	44.1	95.4	96.5	88.9	99.2	98.0	95.6	98.5	95.5	-	-	93.0	98.9	99.0	96.4
2	**0**	-	-	-	-	-	-	-	-	-	-	-	-	93.8	-	-	-	-
	1	-	-	-	-	-	-	-	-	-	-	-	-	95.0	-	-	-	-
	2	-	-	-	-	-	-	-	-	-	-	-	-	84.5	-	-	-	-
	3	-	-	-	-	-	-	-	-	-	-	-	-	66.9	-	-	-	-
3	**0**	-	-	99.1	98.7	99.1	97.3	98.3	99.4	98.2	98.3	97.7	98.2	99.0	-	-	-	-
	1	-	-	99.2	99.1	99.0	98.2	98.5	99.6	98.5	98.6	98.4	98.7	100.0	-	-	-	-
	2	-	-	98.2	98.2	98.5	97.2	97.5	98.4	97.1	97.3	96.6	97.6	96.2	-	-	-	-
	3	-	-	96.3	96.9	95.2	95.9	98.7	99.2	98.1	97.9	97.4	97.8	95.9	-	-	-	-

Dev	**S**	**30**	**31***	**32***	**33***	**34***	**35***	**36**	**37**	**38**	**39**	**40**	**42***	**43**	**45**	**53***	**54**	
All	**0**	94.7	52.0	63.7	98.1	**97.2**	**99.0**	**99.9**	96.6	96.4	**99.2**	92.7	66.0	73.8	**98.9**	**98.7**	95.2	-
	1	**95.4**	55.9	68.5	**98.6**	95.7	98.5	98.9	**96.9**	95.8	97.3	98.4	64.8	74.7	99.1	98.2	96.4	-
	2	95.0	59.2	71.0	96.4	93.2	97.9	97.2	94.7	93.8	97.6	86.7	69.8	80.3	97.3	98.0	95.2	-
	3	93.8	**92.0**	**98.6**	93.1	89.8	95.8	85.4	96.3	**97.8**	90.7	**96.6**	**72.1**	**83.4**	74.0	91.6	**100.0**	-
1	**0**	94.7	44.2	2.9	99.4	98.0	99.0	-	100.0	-	-	-	90.3	74.0	98.8	95.0	-	-
	1	95.4	52.7	17.0	99.4	96.6	98.6	-	100.0	-	-	-	90.2	74.8	99.0	95.8	-	-
	2	95.0	55.8	24.2	99.6	95.6	98.7	-	100.0	-	-	-	89.5	80.3	97.4	94.4	-	-
	3	93.8	98.4	98.8	93.0	89.7	95.3	-	97.7	-	-	-	82.0	83.3	69.5	100.0	-	-
2	**0**	-	66.0	11.5	97.9	95.7	99.6	-	-	-	-	-	-	-	100.0	-	-	-
	1	-	59.4	16.7	97.8	96.1	98.2	-	-	-	-	-	-	-	100.0	-	-	-
	2	-	64.5	25.0	92.3	90.5	91.0	-	-	-	-	-	-	-	96.0	-	-	-
	3	-	73.7	98.7	97.1	90.1	98.3	-	-	-	-	-	-	-	96.5	-	-	-
3	**0**	-	100.0	99.2	96.6	96.0	98.1	99.9	96.2	96.4	99.2	92.7	39.5	0.0	100.0	98.8	95.2	-
	1	-	100.0	99.2	98.3	90.1	100.0	98.9	96.5	95.8	97.3	98.4	36.3	0.0	100.0	98.3	96.4	-
	2	-	100.0	98.9	94.4	84.2	100.0	97.2	94.1	93.8	97.6	86.7	49.3	33.3	100.0	98.1	95.2	-
	3	-	100.0	98.4	89.5	89.5	93.1	85.4	96.2	97.8	90.7	96.6	61.6	33.3	100.0	91.2	100.0	-

**** Determines weeks that have executed training / Bold values indicate the best performance per scenario for each week.***

update mechanism, demonstrating the practicality of FL in IIoT systems constrained by computing power, energy, and bandwidth.

VI. CONCLUSION AND FUTURE WORK

IIoT systems increasingly support intelligent maintenance by combining distributed sensing with edge intelligence. This study demonstrates the feasibility of a TinyFL framework for anomaly detection on highly constrained IIoT devices, such as the ESP32, operating within LoRaWAN-enabled environments. The framework adheres to a Tiny design philosophy, prioritizing a minimal memory footprint, short training times, and efficient communication while maintaining competitive detection accuracy. Despite the limited device memory (less than 512 KB SRAM and 8 MB PSRAM), the proposed approach achieved an average F1-score of 97.46%, comparable to centralized and Flower-based FL implementations, and outperforming standard MLP models. The lightweight AE design, together with optimized preprocessing and data reading routines, enabled per-epoch training times as low as 63 s and LoRaWAN communication intervals of 18.4 h at SF9, confirming the feasibility of model updates under highly constrained network conditions. A key contribution of this work is the TTFL strategy, which enables weekly model updates tailored to industrial conditions. By combining the generalization of an initially transferred centralized model with local adaptation, TTFL provides robust anomaly detection, especially during periods of changing machine behavior.

Although energy consumption during LoRaWAN message transmission was not explicitly measured, it remains an important avenue for future investigation. The effectiveness of the system also depends on selecting the appropriate thresholds for anomalies and ensuring sufficient local training data, underscoring the need for adaptive aggregation and automated threshold optimization to improve generalization and minimize manual adjustment. Future work will focus on deploying the framework in real industrial environments and developing more resilient mechanisms to handle packet loss, communication delays, and network desynchronization without compromising training efficiency or model performance in LoRaWAN-based IIoT systems.

REFERENCES

[1] O. Aouedi et al., "A survey on intelligent Internet of Things: Applications, security, privacy, and future directions," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 2, pp. 1238–1292, Apr. 2025.

[2] P. Dini, L. Diana, A. Elhanashi, and S. Saponara, "Overview of AI-models and tools in embedded IIoT applications," *Electronics*, vol. 13, no. 12, p. 2322, Jun. 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/12/2322/html

[3] M. N. Islam, H. M. M. Jamil, S. A. Pranto, R. K. Das, A. Amin, and A. Khan, "Future industrial applications: Exploring LPWAN-driven IoT protocols," *Sensors*, vol. 24, no. 8, p. 2509, 2024. [Online]. Available: https://www.mdpi.com/1424-8220/24/8/2509

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.

[4] S. C. Setya, G. Krismadinata, and J. Weriza, "Systematic literature review and meta-analysis of microcontroller learning development in the Industry 4.0," *Nanotechnol. Perceptions*, vol. 20, pp. 873–896, 2024.

[5] P. Maurya, A. Hazra, P. Kumari, T. B. Sørensen, and S. K. Das, "A comprehensive survey of data-driven solutions for LoRaWAN: Challenges and future directions," *ACM Trans. Internet Things*, vol. 6, no. 1, pp. 1–36, Feb. 2025, doi: 10.1145/3711953.

[6] E. Dritsas and M. Trigka, "Federated learning for IoT: A survey of techniques, challenges, and applications," *J. Sensor Actuator Netw.*, vol. 14, no. 1, p. 9, Jan. 2025. [Online]. Available: https://www.mdpi.com/2224-2708/14/1/9/html

[7] A. Liso et al., "A review of deep learning-based anomaly detection strategies in industry 4.0 focused on application fields, sensing equipment, and algorithms," *IEEE Access*, vol. 12, pp. 93911–93923, 2024.

[8] S. Trilles, S. S. Hammad, and D. Iskandaryan, "Anomaly detection based on artificial intelligence of things: A systematic literature mapping," *Internet Things*, vol. 25, Apr. 2024, Art. no. 101063. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2542660524000052

[9] R. Tejaswini et al., "Anomaly detection in IoT networks: A deep learning approach using autoencoders," *IEEE Access*, vol. 13, pp. 132739–132749, 2025.

[10] G. Iuhasz, T.-F. Fortiş, and S. Panica, "Exploring machine learning methods for the identification of production cycles and anomaly detection," *Internet Things*, vol. 30, Mar. 2025, Art. no. 101508. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2542660525000216

[11] C. N. da Silva and C. V. S. Prazeres, "Tiny federated learning for constrained sensors: A systematic literature review," *IEEE Sensors Rev.*, vol. 2, pp. 1–16, 2025.

[12] E. Njor, M. A. Hasanpour, J. Madsen, and X. Fafoutis, "A holistic review of the TinyML stack for predictive maintenance," *IEEE Access*, vol. 12, pp. 184861–184882, 2024.

[13] S. R. Annapareddy, "Edge AI for real-time fault detection in embedded systems," *Int. J. Emerg. Trends Comput. Sci. Inf. Technol.*, pp. 235–238, 2025. [Online]. Available: https://www.ijetcsit.org/index.php/ijetcsit/article/view/203

[14] N. X. Long, "Lightweight unsupervised model for anomaly detection on microcontroller platforms — journal of maritime research," *J. MARITIME Res.*, vol. 21, pp. 187–195, Aug. 2024. [Online]. Available: https://www.jmr.unican.es/jmr/article/view/874

[15] F. Lachekhab, M. Benzaoui, S. A. Tadjer, A. Bensmaine, and H. Hamma, "LSTM-autoencoder deep learning model for anomaly detection in electric motor," *Energies*, vol. 17, no. 10, p. 2340, May 2024. [Online]. Available: https://www.mdpi.com/1996-1073/17/10/2340

[16] I. Sharifirad, J. Boudjadar, and P. G. Larsen, "Cascaded TinyML-based reduction for the anomaly detection model of an industrial combustion system," in *Proc. IEEE 8th Int. Conf. Ind. Cyber-Phys. Syst. (ICPS)*, May 2025, pp. 1–6.

[17] B.-S. Kim and S.-K. Ro, "Development of a real-time anomaly detection system for dry vacuum pumps using low-cost IoT devices and machine learning," *Int. J. Precis. Eng. Manuf.*, vol. 25, no. 10, pp. 2133–2143, Oct. 2024. [Online]. Available: https://link.springer.com/article/10.1007/s12541-024-01039-y

[18] M. Harrabi, A. Hamdi, B. Ouni, and J. Bel Hadj Tahar, "Real-time temperature anomaly detection in vaccine refrigeration systems using deep learning on a resource-constrained microcontroller," *Frontiers Artif. Intell.*, vol. 7, Aug. 2024, Art. no. 1429602.

[19] L. S. Martinez-Rau, Y. Zhang, B. Oelmann, and S. Bader, "TinyML anomaly detection for industrial machines with periodic duty cycles," in *Proc. IEEE Sensors Appl. Symp. (SAS)*, Jul. 2024, pp. 1–6.

[20] O. T. Sanchez, G. Borges, D. Raposo, A. Rodrigues, F. Boavida, and J. S. Silva, "Federated learning framework for LoRaWAN-enabled IIoT communication: A case study," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 24944–24957, Jul. 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10954980/

[21] S. M. Hernandez and E. Bulut, "WiFederated: Scalable WiFi sensing using edge-based federated learning," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12628–12640, Jul. 2022.

[22] S. Sebbio, G. Morabito, A. Catalfamo, L. Carnevale, and M. Fazio, "Federated learning on raspberry PI 4: A comprehensive power consumption analysis," in *Proc. IEEE/ACM 16th Int. Conf. Utility Cloud Comput.*, Dec. 2023, pp. 1–6. [Online]. Available: https://dl.acm.org/doi/pdf/10.1145/3603166.3632545

[23] S. Aatab and F. Freitag, "Integrating a neural network library in embedded federated learning," in *Proc. IEEE 6th Int. Conf. Cloud Comput. Artif. Intell., Technol. Appl. (CloudTech)*, Nov. 2023, pp. 01–07.

[24] V. Tsoukas, A. Gkogkidis, E. Boumpa, and A. Kakarountas, "A review on the emerging technology of TinyML," *ACM Comput. Surv.*, vol. 56, no. 10, pp. 1–37, Oct. 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3661820

[25] A. Elhanashi, P. Dini, S. Saponara, and Q. Zheng, "Advancements in TinyML: Applications, limitations, and impact on IoT devices," *Electronics*, vol. 13, no. 17, p. 3562, Sep. 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/17/3562

[26] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, "Federated learning for IoT devices: Enhancing TinyML with on-board training," *Inf. Fusion*, vol. 104, Apr. 2024, Art. no. 102189.

[27] N. Llisterri Giménez, M. Monfort Grau, R. Pueyo Centelles, and F. Freitag, "On-device training of machine learning models on microcontrollers with federated learning," *Electronics*, vol. 11, no. 4, p. 573, Feb. 2022. [Online]. Available: https://www.mdpi.com/2079-9292/11/4/573/html

[28] N. L. Giménez, F. Freitag, L. Navarro, and M. Selimi, "Demo: Edge federated learning over a LoRa mesh network," in *Proc. IEEE/ACM Symp. Edge Comput. (SEC)*, Dec. 2024, pp. 510–511.

[29] L. Wulfert, C. Wiede, and A. Grabmaier, "TinyFL: On-device training, communication and aggregation on a microcontroller for federated learning," in *Proc. 21st IEEE Interregional NEWCAS Conf. (NEWCAS)*, Jun. 2023, pp. 1–5.

[30] H. Cheng, Q. Chen, Y. Liang, and G. Zhu, "TinyFL: A lightweight federated learning method with efficient memory-and-communication," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2023, pp. 608–613.

[31] L. Wulfert, N. Asadi, W.-Y. Chung, C. Wiede, and A. Grabmaier, "Adaptive decentralized federated gossip learning for resource-constrained IoT devices," in *Proc. 4th Int. Workshop Distrib. Mach. Learn.*, Dec. 2023, pp. 27–33, doi: 10.1145/3630048.3630181.

[32] N. L. Giménez, J. M. Solé, and F. Freitag, "Embedded federated learning over a LoRa mesh network," *Pervas. Mobile Comput.*, vol. 93, Jun. 2023, Art. no. 101819.

[33] S. Aggarwal and A. Nasipuri, "FL-LoRaMAC: A novel framework for enabling on-device learning for LoRa-based IoT applications," *Future Internet*, vol. 15, no. 9, p. 307, Sep. 2023. [Online]. Available: https://www.mdpi.com/1999-5903/15/9/307

[34] S. S. Borkotoky, "Balancing the energy consumption and latency of over-the-air firmware updates in LoRaWAN," *IEEE Trans. Ind. Informat.*, vol. 21, no. 10, pp. 7403–7411, Oct. 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11045540

[35] P. Manikandan, "A technical comparative analysis of LoRaWAN technologies for smart industrial IoT," in *Proc. Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, Mar. 2025, pp. 103–108. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10984929

[36] D. Zorbas and A. Sabyrbek, "On the uplink repetitions in LoRaWAN," *IEEE Netw. Lett.*, vol. 7, no. 4, pp. 342–346, Dec. 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11204482

[37] W. Guo, F. Zhuang, X. Zhang, Y. Tong, and J. Dong, "A comprehensive survey of federated transfer learning: Challenges, methods and applications," *Frontiers Comput. Sci.*, vol. 18, no. 6, pp. 1–34, Dec. 2024. [Online]. Available: https://link.springer.com/article/10.1007/s11704-024-40065-x

[38] R. Tang and M. Jiang, "Enhancing federated learning: Transfer learning insights," in *Proc. IEEE 3rd Int. Conf. Electr. Eng., Big Data Algorithms (EEBDA)*, Feb. 2024, pp. 1358–1362.

[39] R. Haripriya et al., "Optimizing fault tolerance and latency of federated learning using edge servers and pre-trained model," *IEEE Access*, vol. 13, pp. 158737–158750, 2025.

[40] Y. N. Tan, V. P. Tinh, P. D. Lam, N. H. Nam, and T. A. Khoa, "A transfer learning approach to breast cancer classification in a federated learning framework," *IEEE Access*, vol. 11, pp. 27462–27476, 2023.

[41] Y. Tang, S. Zhang, and Z. Zhang, "A privacy-preserving framework integrating federated learning and transfer learning for wind power forecasting," *Energy*, vol. 286, Jan. 2024, Art. no. 129639. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0360544223030335

[42] O. T. Sanchez, G. Borges, D. Raposo, A. Rodrigues, F. Boavida, and J. S. Silva, "Enhancing the performance of industrial IoT LoRaWAN-enabled federated learning frameworks: A case study," *Internet Things*, vol. 32, Jul. 2025, Art. no. 101632. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2542660525001465

[43] L. V. Winkle. (2019). *Codeplea/GENANN: Simple Neural Network Library in ANSI C*. [Online]. Available: https://github.com/codeplea/genann

Authorized licensed use limited to: BRAC UNIVERSITY. Downloaded on October 03,2026 at 13:30:43 UTC from IEEE Xplore. Restrictions apply.